GraphSynth: Agentic Triton Kernel Synthesis for Non-Standard Attention, Verified Across GPU Generations

Joshi Penta, Priyesh Shukla

International Institute of Information Technology, Hyderabad · Nov 01 2026, Athens, Greece

Know more about ongoing works at **SARCS Research Group**

IEEE/ACM International Symposium on Microarchitecture®

A³ | AGENTIC APPROACHES TO ARCHITECTURE WORKSHOP @ MICRO 2026

TL;DR

An LLM writes the missing Triton kernels; a gate it **cannot reach** verifies and times them. **10/10** variants, **10.82×** over eager PyTorch.

MOTIVATION

The gap.
Modern models do not use standard causal softmax attention. They use sigmoid or ReLU normalisation, logit soft-capping, positional biases and structured masks. For these, **no fused kernel exists**.

The cost.
PyTorch falls back to eager and writes the full $S\times S$ scores to DRAM at every layer.

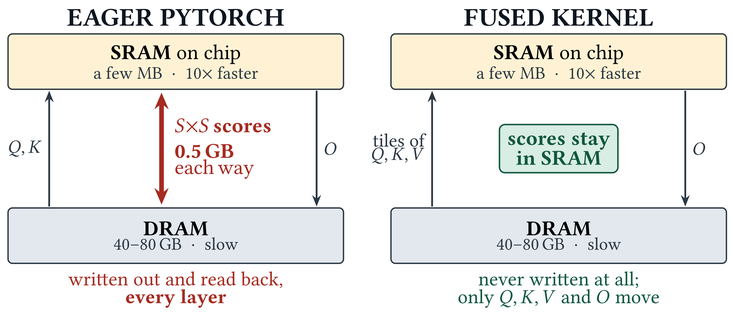
The risk.
An LLM can write the missing kernel. But if it can reach the correctness test or the timer, **editing an assertion is easier than writing a fast kernel.**

	blk-diag	sld-win	ReLU	local	ALiBi	softcap	sigmoid	prefix	dilated	per-head
eager PyTorch	×	×	×	×	×	×	×	×	×	×
`torch.compile`	~	~	~	~	~	~	~	~	~	~
FlashAttention	×	×	×	×	×	×	×	×	×	×
FlexAttention	✓	✓	×	✓	✓	✓	×	✓	✓	✓
GraphSynth	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

✓ fused kernel ~ partial (≈4× eager) × none: scores go to DRAM

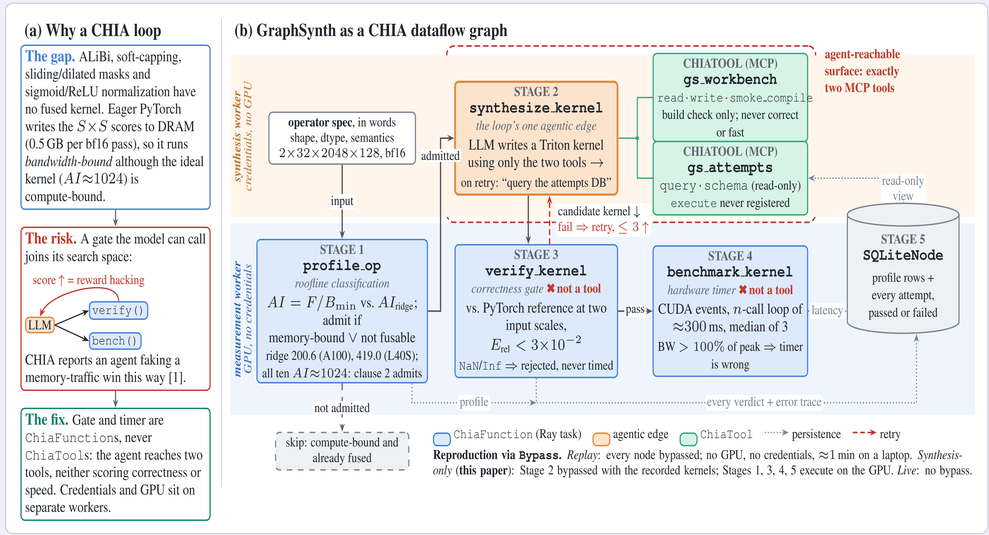
Ten attention variants against every fused path PyTorch ships. **FlashAttention covers none of them**; FlexAttention cannot express sigmoid or ReLU attention at all. GraphSynth produced a verified kernel for all ten.

WHY IT IS SLOW



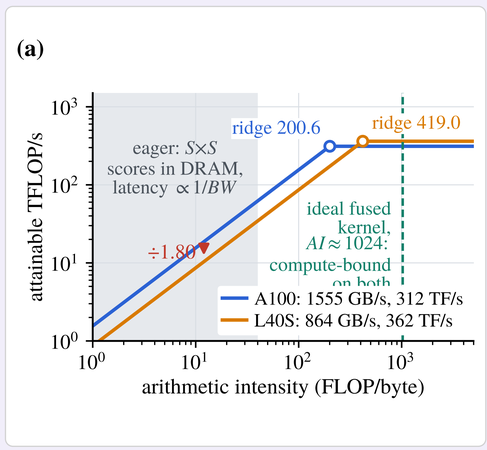
The arithmetic is **compute-bound** ($AI \approx 1024$, far past the A100 ridge point of 200.6). The *execution* is **bandwidth-bound**. That gap is the whole problem.

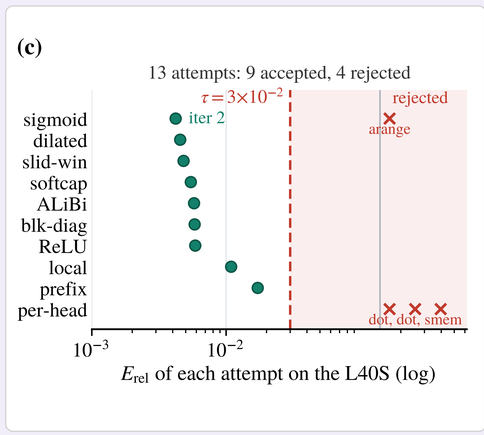
THE LOOP



Five stages run in order. **Only Stage 2 is the LLM**. The dashed red box is everything it can touch. The gate and the timer sit outside it.

CLASSIFICATION, AND THE GATE





(a) Stage 1: the ridge point doubles, 200.6 → 419.0, yet all ten still admit. **(c) Stage 3:** every attempt against the gate: **9 accepted, 4 rejected** in 13. Stage 2 never sees this test.

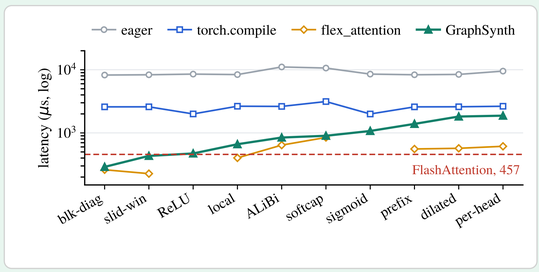
REPRODUCE EVERY NUMBER

Project page is live at
`joship-commits.github.io/graphsynth`

Code, kernels and every CSV
`github.com/JoshiP-commits/CHIA_HACKATHON`

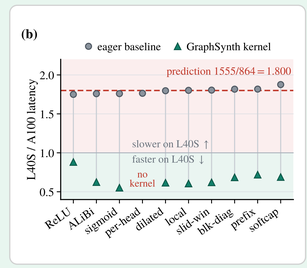
One command replays a recorded run in about a minute on a laptop, with **no GPU and no API key**.

RESULTS ON THE A100



Lower is better. A verified kernel for **all ten** variants: 10.82× over eager PyTorch, 3.04× over `torch.compile`. Two kernels beat PyTorch's fused FlashAttention path.

CROSS-HARDWARE VALIDATION



The L40S has 1.8× less DRAM bandwidth. Eager PyTorch slowed by **1.795**×, almost exactly the **1.800**× bandwidth drop. GraphSynth's kernels ran *faster*, reaching **32.02**× over eager: they bypass the DRAM bottleneck entirely.

Acknowledgements. This work began at the CHIA Hackathon. The A100 and L40S experiments were run on Google Cloud, supported by Google Cloud research credits.